

Sample Solution: 1 (20%)

- (a). (5%) -- application, transportation, network, data link, physical
- (b). (2%) -- network
- (c). (2%) -- transportation
- (d). (2%) -- data link
- (e). (2%) -- e.g. TCP, UDP
- (f). (2%) -- e.g. http, smtp, dns
- (g). (2%) -- e.g. packet error, packet drop due to queue overflow
- (h). (3%) -- all

Sample Solution: 2 (15%)

Sample Solution: 2 (15%)

(a). (2%)-- d_{proc} : e.g. time for error detection, extracting destination ip address

(2%)-- d_{queue} : time for waiting those packets waiting ahead to be sent out

(2%)-- d_{trans} : time for push the packet out = L/R (L : packet length, R : network link bandwidth)

(2%)-- d_{prop} : time for bit traveling from one end to the other = D/V (D : distance between the two ends, V : signal propagation speed)

(b). (3%) queuing delay = $10 * (\frac{1000 * 8 (bit)}{1 * 10^3 (b/s)}) = 8 * 10^{-2} (sec)$

(2%) transmission delay = $\frac{1000 * 8 (bit)}{1 * 10^3 (b/s)} = 8 * 10^{-6} (sec)$

(2%) propagation delay = $\frac{1000 * 10^3 (m)}{2 * 10^8 (m/s)} = 5 * 10^{-3} (sec)$

Sample Solution: 3 (11%)

Sample Solution: 3 (11%)

- (a). (5%)-- (a),(c)
- (b). (2%)-- ref. circuit switch would reserve bandwidth in advance for each possible connections, no matter in the end the connection is active or not; on the other hand, packet switching utilizes statistical multiplexing to utilize available bandwidth at risk of packet contention.
- (c). (2%)-- ref. circuit switching tends to reserve bandwidth for a connection, which potentially reduces chances of packet loss and is good for real-time communication applications such as audio/video transmission.
- (d). (2%)-- ref. "store-and-forward" transmission scheme means packet should not being forwarded to next node before all bits of the packet arriving completely at current node.

Sample Solution: 4 (16%)

- (a). (2%)-- ref. domain name resolving (domain name to IP mapping)
- (b). (2%)-- ref. the IP addresses for root DNS servers are given (hardcoded) to local name servers during software installation.
- (c). (2%)-- ref. 1. Avoid single point of failure (SPF), 2. It is not scalable
- (d). (2%)-- ref. iterative mechanism switches most of the name resolving task workloads to local name servers, and so effectively reduce the burden of global name servers.
- (e). (2%)-- ref. Since name resolving results are cached and won't do it again before catch time expired.
- (f). (2%)-- UDP
- (g). (4%)-- (i) (b) MX; (ii) (d) CNAME; (iii) (a) A; (iv) (c) NS

Sample Solution: 5 (18%)

Sample Solution: 5 (18%)

- (a). (9%)-- $\{1,2\} = \{\text{corrupt}(\text{rcvpkt}), \text{isACK}(\text{rcvpkt}, 1)\}$, $\{3,4\} = \{\text{udt_send}(\text{sndpkt}), \text{start_timer}\}$, $5 = \text{notcorrupt}(\text{rcvpkt})$, $6 = \text{sndpkt} = \text{make_pkt}(1, \text{data}, \text{checksum})$, $7 = \text{timeout}$, $\{8,9\} = \{\text{notcorrupt}(\text{rcvpkt}), \text{isACK}(\text{rcvpkt}, 1)\}$
- (b). (3%)-- $A^0 A^1 B^0 A^1 B^0 A^1 C^0 C^1 C^0$
- (c). (3%)-- ref. the time sender busy sending by definition is L/R ;
total time spend before to send next packet by definition is “the time to sending” + “the time to wait for ack” = $L/R + RTT$
- (d). (3%)-- ref. it is impossible. According to the protocol the sender will keep waiting for the right ack coming back before it sends the next packet. So, it is impossible for the case of back-to-back packet loss.

Sample Solution: 6 (9%)

--ref (c), (f)

start_timer}, 5=notcorrupt(rcvpkt), 6=sendpkt=make_pkt(1, data, checksum), 7=timeout, {8,9}={notcorrupt(rcvpkt),isACK(rcvpkt,1)}

(b). (3%)-- $A^0 A^1 B^0 A^1 B^0 A^1 C^0 C^1 C^0$

(c). (3%)-- ref. the time sender busy sending by definition is L/R ;

total time spend before to send next packet by definition is "the time to sending" + "the time to wait for ack" = $L/R + RTT$

(d). (3%)-- ref. it is impossible. According to the protocol the sender will keep waiting for the right ack coming back before it sends the next packet. So, it is impossible for the case of back-to-back packet loss.

Sample Solution: 6 (9%)

--ref (c), (f)

Sample Solution: 7 (6%)

-- ref. (?1) (B), (?2) (C), (?3) (A), (?4) (F), (?5) (E), (?6) (D),